

MODULE 5: SCALABLE ML(ONLINE & DISTRIBUTED)

Contents

Scalability in Machine Learning	1
Online Learning in Machine Learning.....	2
Distributed Learning	3
Core Paradigms of Distributed Learning	3
Semi-Supervised Learning (SSL).....	4
Active Learning	5
Reinforcement Learning (RL)	6
Inference in Graphical Models.....	7
Types of Inference	8
Exact Inference Methods.....	8
Approximate Inference Algorithms	9

Scalability in Machine Learning

Scalability in ML refers to the ability of a machine learning system (including the algorithms, hardware, and infrastructure) to handle a rapidly **increasing volume, velocity, and variety of data** (the "Big Data V's") and a rapidly **increasing complexity** of models (e.g., larger neural networks).

Why Scalability is Needed

1. **Massive Data Growth:** The sheer volume of data being generated globally (from sensors, social media, IoT, etc.) far exceeds what a single machine can process. Models must scale to consume terabytes or petabytes of data.
2. **Model Performance:** More data generally leads to **better, more accurate models**. Scaling allows models to learn from larger datasets, capturing subtle patterns that small samples miss.
3. **Real-Time Requirements:** Many applications (e.g., fraud detection, recommendation engines) require models to process streams of data and make predictions with **very low latency** (high velocity).
4. **Complex Models:** Modern deep learning architectures (like large language models or deep CNNs) have billions of parameters and require massive computational power (GPUs/TPUs) and efficient distributed training frameworks.
5. **Cost Efficiency:** Scaling horizontally (using many commodity machines) is often much more cost-effective than scaling vertically (upgrading a single powerful machine).

Big Data Challenges (The 5 V's)

1. **Volume:** The sheer **scale** of data being generated (Terabytes, Petabytes, Exabytes) makes it impossible for single-machine processing. This requires distributed storage and parallel processing architectures.
2. **Velocity:** The **speed** at which data is generated, collected, and must be processed (often in real-time streams) demands low-latency infrastructure and continuous model updating.

3. **Variety:** The diversity of data formats, sources, and types (structured tables, unstructured text, sensor readings, video, audio) requires complex pre-processing and flexible, multi-modal algorithms.
4. **Veracity:** The **quality, accuracy, and trustworthiness** of the data. Big Data is often noisy, incomplete, inconsistent, or biased, requiring robust validation and cleaning pipelines.
5. **Value:** The challenge of extracting **meaningful insights and business value** from the massive datasets. If the system is not scalable, the time and cost to process the data will negate any potential value.

Online Learning in Machine Learning

Online Learning is a machine learning method where the model is trained sequentially as new data becomes available, one instance or a small batch at a time. Unlike **Batch Learning** (or Offline Learning), where the model is trained once on the entire dataset, online learning continuously updates the model's parameters.

This approach is essential for scenarios involving **data streams** and **concept drift**.

1. **Data Flow:** Data arrives sequentially as a stream; it is processed immediately and typically discarded (or archived) afterward.
2. **Model Update:** The model parameters are updated **incrementally** after processing each new data point or small mini-batch.
3. **Memory:** Requires **low memory** as it doesn't need to load the entire dataset into memory simultaneously.
4. **Speed/Latency:** Generally, **faster** for prediction and updating, making it suitable for real-time systems.
5. **Adaptability:** Highly effective at adapting to **concept drift** (where the statistical properties of the target variable change over time).

Why Online Learning Is Needed

- In many real-world systems, data arrives continuously and cannot be stored or processed all at once (e.g., stock prices, sensor streams, online clicks).
- Traditional batch learning becomes slow, expensive, and impractical as data grows.
- Online learning makes it possible to:
 - update models quickly,
 - respond to new information instantly,
 - and adapt to changes in data distribution (concept drift).

Examples of Online Learning Algorithms

Many common algorithms can be adapted for online learning by using **Stochastic Gradient Descent (SGD)** or its variants, which update weights based on one or a few samples at a time:

- **Perceptron/Stochastic Gradient Descent (SGD):** The fundamental optimizer used to update weights in neural networks and linear models in an online fashion.
- **Online SVM:** Uses techniques like SGD to update the support vectors incrementally.
- **Adaptive Boosting (AdaBoost):** Can be adapted for online scenarios (e.g., Online AdaBoost) to handle data streams.
- **Streaming Algorithms:** Specialized algorithms (like some clustering or decision tree algorithms) built to process single passes of data.

Aspect	Online Learning	Batch Learning (Offline)
Training Mode	Sequential and incremental.	Static, on the complete dataset.
Resource Usage	Low memory, high iteration speed.	High memory, low iteration speed.
Adaptability	High; adapts to changes in the data distribution.	Low; requires retraining on the full dataset to adapt.
Use Cases	Stock market prediction, spam filtering, click-through rate prediction.	Image classification, Natural Language Processing (NLP) models.

Distributed Learning

Distributed Learning in Machine Learning (ML) refers to the training of a model across **multiple computational units** (CPUs, GPUs, TPUs, or even different machines/nodes) connected over a network. This is a crucial technique for addressing the **Big Data challenges** of Volume and Velocity, allowing models to scale beyond the limitations of a single server.

Why Use Distributed Learning?

1. **Scaling Data (Big Volume):** It allows training on datasets too large to fit into the memory of a single machine (e.g., Terabytes or Petabytes).
2. **Scaling Model Complexity:** It enables the use of extremely large, complex models (like large language models with billions of parameters) by distributing the model's structure across multiple devices.
3. **Speed/Throughput (High Velocity):** It drastically reduces training time by performing computations in parallel.

Core Paradigms of Distributed Learning

Distributed learning primarily operates under two main paradigms, often differentiated by what is distributed: the data or the model.

1. Data Parallelism: This is the **most common** approach, used when the model fits onto a single device, but the dataset is too large or training is too slow.

- **Mechanism:** The **model** is replicated on every worker node. The **data** is split into different partitions, and each worker receives a distinct subset.
- **Training Flow:**
 1. Each worker calculates the gradient based on its local subset of data.
 2. The workers send their local gradients to a central **Parameter Server** (or aggregate them directly, like in **All-Reduce**).
 3. The gradients are averaged, and the resulting aggregated update is sent back to all workers to synchronize their model copies.
- **Advantage:** Simple to implement and guarantees convergence (given proper synchronization).

2. Model Parallelism: This is used when the model itself (e.g., a very deep neural network) is **too large** to fit into the memory of a single GPU or CPU.

- **Mechanism:** The **model's layers** or parts are partitioned and distributed across different worker nodes. The **data** (or mini-batch) typically flows sequentially through these partitioned model segments.
- **Training Flow:** Data is passed through the first part of the model on one worker, the output is passed to the next worker for the next segment of the model, and so on.
- **Advantage:** Necessary for training the largest models that cannot be fit otherwise.

Applications of Distributed Learning

- Training large NLP models (BERT, GPT)
- Large-scale recommendation systems (YouTube, Amazon)
- Fraud detection systems processing billions of transactions
- Search ranking (Google, Bing)
- Self-driving car sensor data training

Semi-Supervised Learning (SSL)

Semi-Supervised Learning (SSL) is a machine learning paradigm that utilizes a training dataset containing a **small amount of labeled data** and a **large amount of unlabeled data**.

SSL aims to leverage the information contained within the vast quantity of unlabeled data to improve the accuracy of models, which would otherwise be limited by only training on the small labeled set.

Why Use Semi-Supervised Learning?

1. **Labeling Cost:** Obtaining large, high-quality labeled datasets is often expensive, time-consuming, and requires specialized human expertise (e.g., medical image diagnosis, transcribing speech).
2. **Data Abundance:** In most real-world scenarios (images on the internet, text documents), unlabeled data is abundant and cheap to acquire.
3. **Improved Performance:** By understanding the underlying structure of the data's distribution using unlabeled samples, SSL algorithms can often achieve performance comparable to, or sometimes better than, purely supervised methods using only a fraction of the labels.

Core Assumptions in SSL

SSL algorithms rely on the following key assumptions about the data structure to infer labels:

1. **The Smoothness Assumption:** If two data points ($\mathbf{x}_1, \mathbf{x}_2$) are close to each other in the input space, then their corresponding output labels (y_1, y_2) should also be similar (smooth).
2. **The Cluster Assumption:** If data points fall into distinct clusters, the points within the same cluster are likely to belong to the same class. The decision boundary should lie in low-density regions (between clusters).
3. **The Manifold Assumption:** The high-dimensional data actually lies on a lower-dimensional structure (a manifold). Learning the structure of this manifold from unlabeled data helps generalize the learned labels.

Popular Semi-Supervised Learning Methods

1. Self-Training

- a. The model is first trained on labeled data.
- b. Then it predicts labels for unlabeled data.
- c. Confident predictions are added back into the training set.
- d. Simple and widely used.

2. Co-Training

- a. Two classifiers are trained on different views of the same data (e.g., text vs. hyperlinks).
- b. Each classifier labels unlabeled data to help the other improve.
- c. Works well when features are naturally split.

3. Graph-Based Methods

- a. Data is represented as a graph with nodes (samples) and edges (similarities).
- b. Labels propagate from labeled points to unlabeled ones over the graph structure.
- c. Used in social networks, NLP, image clustering.

4. Semi-Supervised SVM (S3VM)

- a. Finds a decision boundary that not only separates labeled data but also avoids cutting through high-density regions of unlabeled data.

5. Neural Network-Based SSL

- a. Modern SSL uses pseudo-labeling, consistency loss, and techniques like FixMatch, MixMatch, etc.
- b. Very effective for image and text tasks.

Advantages of Semi-Supervised Learning

- Improves performance when labeled data is scarce
- Reduces annotation cost and dependence on experts
- Uses the structure of unlabeled data to enhance generalization
- Useful for real-world datasets where labels are difficult to obtain

Limitations

- Incorrect pseudo-labels can mislead the model
- Assumptions (cluster, smoothness) may not always hold
- Performance depends heavily on the quality of unlabeled data

Active Learning

Active Learning is a specialized sub-field of Machine Learning where the learning algorithm is allowed to **query** an oracle (usually a human annotator) to obtain the desired output (label) for new, unlabeled data points.

Instead of passively accepting a fixed, pre-labeled dataset, the active learner strategically **selects the most informative examples** from a large pool of unlabeled data to be labeled.

Why Use Active Learning?

Active learning addresses the core problem of **labeling cost**.

- **Maximizing Label Efficiency:** The goal is to achieve the highest possible model accuracy while using the **minimum number of labels**.

- **Limited Budget:** When the budget for human annotation is severely constrained, active learning ensures that every dollar/label spent yields the maximum improvement in model performance.

The Active Learning Cycle

Active learning is an iterative, closed-loop process that repeats until the labeling budget is exhausted or the desired accuracy is reached:

1. **Start:** The model is trained on a small initial labeled dataset.
2. **Query Strategy:** The algorithm selects a batch of the **most informative** unlabeled data points from the pool.
3. **Oracle Query:** The selected data points are sent to the human annotator (the oracle) for labeling.
4. **Enrichment:** The new labels are added to the labeled training set.
5. **Retrain:** The model is retrained on the augmented labeled dataset, and the process repeats.

Advantages of Active Learning

- Reduces labeling cost dramatically
- Improves accuracy even with limited labeled data
- Focuses annotation effort where it matters most
- Useful for domains requiring expert labeling

Limitations

- Incorrect selection strategies can mislead the model
- Requires human-in-the-loop setup
- Computationally expensive for very large pools
- Performance depends on the quality of uncertainty estimates

Reinforcement Learning (RL)

Reinforcement Learning (RL) is an area of Machine Learning where an **Agent** learns how to behave in an **Environment** by performing **Actions** and receiving **Rewards** or **Penalties**. The goal of the agent is to learn an optimal **Policy** that maximizes the cumulative reward over time.

Core Components of RL

RL is typically modeled as a **Markov Decision Process (MDP)** and involves the following five core elements:

1. **Agent:** The learner or decision-maker (e.g., an AI controller, a robot).
2. **Environment:** The world the agent interacts with (e.g., a game board, a physical space).
3. **State (S):** A complete description of the current situation of the agent and the environment (e.g., the position of all pieces on a chess board).
4. **Action (A):** The set of moves or decisions the agent can take in a given state (e.g., move left, jump, choose an advertisement).
5. **Reward (R):** A scalar feedback signal that the environment gives to the agent after each action. The reward defines the goal of the problem (e.g., +10 points for winning, -1 for hitting a wall).

Key Concepts and Goal

- **Policy (π):** The agent's brain; it is the function that maps a **state (S) to an action (A)**. The entire objective of RL is to find the **optimal policy (π^*)** that maximizes the expected total reward.
- **Value Function (V):** Predicts the **expected future reward** an agent will receive starting from a certain state and following a specific policy. It tells the agent "how good" a state is.
- **Q-Function (Q):** Predicts the expected future reward for taking a specific **Action (A) in a specific State (S)** and then following the policy afterwards. It tells the agent "how good" a state-action pair is.

Exploration vs. Exploitation

A fundamental challenge in RL is balancing **Exploration** and **Exploitation**:

- **Exploitation:** The agent uses its current knowledge (the best known policy) to choose the action it believes will yield the highest immediate reward.
- **Exploration:** The agent chooses a random or suboptimal action to discover new information about the environment, which might lead to higher rewards in the future.

Common strategies to balance this include the **ϵ -greedy** method, where the agent explores randomly with a small probability (ϵ) and exploits otherwise.

Types of RL Methods

a) Value-Based Methods

- Learn value functions and choose actions based on them.
- The most famous example is Q-Learning, which learns a Q-table mapping:
 - $Q(s,a) \rightarrow$ expected future reward

b) Policy-Based Methods

- Directly learn a policy without computing value functions.
- Useful for continuous action spaces (robotics).

c) Actor-Critic Methods

- Combine both approaches.
- Actor learns the policy; Critic learns the value.
- More stable and widely used in deep RL (A2C, A3C).

Inference in Graphical Models

Graphical Models

- Graphical Models are probabilistic models that represent variables and their dependencies using graphs.
- Nodes represent random variables, and edges represent probabilistic relationships.
- They provide a compact, visual, and mathematically elegant way to model complex joint distributions
- Two major types:
 - **Bayesian Networks (Directed Graphs)**
 - **Markov Random Fields (Undirected Graphs)**

Inference in Graphical Models

- Inference means computing probabilities or finding the most likely values of variables in the model. Typical tasks include:

- Computing the probability of some event
- Finding the most probable explanation given evidence
- Predicting unknown variables from known ones
- In short: Inference is about answering “What is the probability of X, given Y?” using the graphical structure.

Types of Inference

1. **Marginal Inference: Goal:** To find the **unconditional probability distribution** of a variable or a subset of variables, ignoring all other variables in the model.

- **Calculation:** It is computed by summing (or integrating) the **joint probability distribution** over all possible values of the variables that are *not* of interest. This process is called **marginalization**.

- Formula (for discrete variables A and B):

$$P(A) = \sum_B P(A, B)$$

- **Use Case:** Answering simple probability questions like, "What is the overall probability that the sprinkler is on?"

2. **Conditional Inference: Goal:** To find the **probability distribution** of one set of variables (the query variables) **given the observed values** of another set of variables (the evidence).

- **Calculation:** This is computed using the definition of conditional probability, which reduces the problem to a ratio of two marginalization problems.

- Formula (for query X and evidence E):

$$P(X|E = e) = \frac{P(X, E = e)}{P(E = e)} = \alpha \sum_Z P(X, E = e, Z)$$

(Where Z is the set of all unobserved variables, and α is a normalization constant).

- **Use Case:** Answering updated probability questions like, "What is the probability of grass being wet **given that** the sprinkler was on?"

3. **MAP Inference (Maximum A Posteriori): Goal:** To find the **single most probable assignment** (the configuration of values) for a set of unobserved variables, given the evidence.

- **Nature:** Unlike marginal and conditional inference, which involve **summation** (integration), MAP inference is an **optimization** problem involving **maximization**.

- Formula (for variables of interest W and evidence E):

$$\mathbf{W}_{\text{MAP}} = \underset{\mathbf{w}}{\operatorname{argmax}} P(\mathbf{W} = \mathbf{w} | E = e)$$

- **Use Case:** Making a **decision** or prediction, such as, "Given the symptoms (evidence), what is the single most likely disease (assignment of W to a disease)?"

Exact Inference Methods

These algorithms guarantee the exact probability but can be computationally expensive (intractable or NP-hard) for large, complex graphs with many loops.

4. **Variable Elimination (VE):**

- **Mechanism:** Systematically eliminates unobserved, non-query variables one by one by pushing the summations/integrals inside the product of factors (local probability distributions).

- **Limitation:** Computation time is exponential in the size of the **largest intermediate factor** created (the *treewidth* of the graph).

5. Belief Propagation (BP) / Sum-Product Algorithm:

- **Mechanism:** An iterative message-passing algorithm where each node in the graph sends messages (representing beliefs or influence) to its neighbors. The process continues until messages converge.
- **Usage:** Guarantees exact results for **tree-structured** or **singly-connected** graphs (graphs with no loops). For graphs with loops (loopy belief propagation), it is often used as a fast, effective **approximate** method, though convergence is not guaranteed.

6. Junction Tree Algorithm:

- **Mechanism:** Converts any arbitrary graph into a special graph structure called a **Junction Tree** (or Clique Tree) and then runs the Belief Propagation algorithm on this tree.
- **Usage:** Guarantees exact inference for any graph structure, but the conversion (triangulation) complexity is exponential in the size of the largest clique, making it slow for dense graphs.

Approximate Inference Algorithms

When exact inference is intractable, approximate methods are used to estimate the probabilities, trading accuracy for speed.

1. Sampling Methods (Monte Carlo):

- **Mechanism:** Generates a large number of random samples from the probability distribution and uses the frequency of those samples to estimate the desired marginals or conditional probabilities.
- **Examples:** **Importance Sampling**, **Markov Chain Monte Carlo (MCMC)**, and **Gibbs Sampling**.

2. Variational Methods:

- **Mechanism:** Transforms the inference problem into an optimization problem. It attempts to find a simpler, tractable probability distribution Q that is "closest" to the true, intractable distribution P .
- **Examples:** **Mean-Field Approximation** (assumes all variables are independent to simplify the graph structure).

Why Inference Is Hard

1. **Exponential Complexity:** Exact inference requires summing over an exponential number of possible assignments (2^N for N binary variables), making it **NP-hard** for general graphs.
2. **Intractable Normalization:** Calculating the **Partition Function (Z)** (the normalization constant) is itself an intractable marginalization problem.
3. **Graph Cycles (Loops):** The presence of loops prevents simple, exact algorithms like Belief Propagation from working correctly, forcing the use of complex, slower methods like the Junction Tree algorithm.
4. **Local Maxima in MAP:** For **Maximum A Posteriori (MAP)** inference, finding the single most likely assignment is often an optimization problem with many local optima.
5. **High Treewidth:** The performance of many exact algorithms (like Variable Elimination) scales poorly with the **treewidth** of the graph, which measures its structural complexity.

Applications of Inference

1. **Sequence Labeling (NLP):** Inferring the most likely hidden sequence (e.g., Part-of-Speech tags, Named Entities) given a sentence using **CRFs/HMMs**.
2. **Computer Vision:** Performing image tasks like **segmentation, denoising, and restoration** by inferring pixel labels using **MRFs**.
3. **Error Correction: Decoding noisy signals** in communication systems to infer the originally transmitted data bits.
4. **Medical Diagnosis:** Inferring the probability of a specific **disease or cause** (hidden state) given a set of symptoms (observed data) using **Bayesian Networks**.
5. **Causal Reasoning:** Modeling complex systems to infer the most likely **causal relationships** and predict outcomes under intervention.